

Concurrent Order Processing

Technical Lead

Working time: 2-4 hours. Stop after 4 hours and note unfinished work.

Deadline: Submit within 24 hours of receiving this assessment.

Stack: ASP.NET Core, EF Core, SQLite or SQL Server, and Angular.

Build an order service that prevents overselling, handles duplicate requests and recovers from notification failures. Use one application, one database and a minimal Angular screen.

Data and endpoints

- Seed two products with a code, price and available quantity. Use decimal values for money.
- POST /api/orders accepts an Idempotency-Key header, a customer reference and line items containing product code and positive quantity. Read prices from the database and calculate the total on the server.
- GET /api/orders/{id} returns the order, total, status and notification status.
- POST /api/orders/{id}/cancel cancels a confirmed order and restores its stock exactly once.
- GET /api/products returns current prices and available quantities.

Concurrency and consistency

- Creating an order must atomically deduct stock, save the confirmed order and create a pending notification. Reject insufficient stock with 409 and leave no partial changes.
- Concurrent orders must never reduce stock below zero. Do not rely on a process-local lock; the database must enforce correctness across application instances.
- Repeating a successful submission with the same key and equivalent payload returns the original order. Reusing that key with a different payload returns 409. Define payload equivalence in the README.
- Concurrent submissions with the same key must create one order and deduct stock once. Document the response given to a request while its duplicate is still processing.
- Concurrent or repeated cancellation must restore inventory once and leave the order Cancelled. A later replay of the original submission must not create or confirm it again.

Notification worker

- Process pending order-created notifications with a background worker and a fake delivery service. Make transient failure configurable.
- Retry failed delivery with bounded attempts and backoff. Mark exhausted notifications as failed and expose their status through the order API.
- Persist pending work so delivery resumes after restart. Use a stable event ID across retries; document the send-success / worker-crash case and any duplicate-delivery risk.

Verification & submission

Concurrent Order Processing

Angular screen

Provide a simple order form, order lookup, cancellation action and notification-status display. Keep the same idempotency key when retrying an uncertain submission; generate a new key for a new order. Show validation, stock-conflict and delivery-failure states.

Required automated tests

Use a real relational database for concurrency and transaction tests. An EF Core in-memory provider is insufficient.

- With one unit in stock, submit two different orders concurrently. Exactly one succeeds, one receives a stock conflict, and stock finishes at zero.
- Submit the same order concurrently with the same key. Verify one order, one stock deduction and one order-created notification record.
- Reuse a successful key with a changed quantity. Verify 409 and no additional database changes.
- Cancel one order concurrently twice. Verify the final status and a single stock restoration.
- Force a failure before the order transaction commits. Verify stock, order and notification changes all roll back.
- Make delivery fail and then succeed. Verify retry behavior and eventual delivery; also verify failed status after attempts are exhausted.

Design note

Keep this to one page. Explain:

- Your transaction boundary, concurrency mechanism, database constraints and idempotency lifecycle.
- How you would safely run multiple notification workers, including work claiming, worker crashes and duplicate delivery. This extension is design-only.
- How you would add a real payment provider without keeping a database transaction open during the network call, including payment success followed by application failure. This extension is design-only.
- The minimum logs and metrics needed to detect stock conflicts, duplicate requests and stuck notifications.

Submit

Provide source code and a README with setup, seed and test commands, assumptions, time spent and unfinished work. No deployment, authentication, real payment integration or message broker is required.

Tools

Documentation and AI tools are allowed. Disclose AI use and be ready to explain your code.